

Lecture 15 – Continuations (2)

COSE212: Programming Languages

Jihyeok Park



2026 Fall

- We will learn about **continuations** with the following topics:
 - **Continuations** (Lecture 14 & 15)
 - **First-Class Continuations** (Lecture 16)
 - **Compiling with continuations** (Lecture 17)
- A **continuation** represents the **rest of the computation**.
 - Continuation Passing Style (CPS)
 - Interpreter of FAE in CPS

- We will learn about **continuations** with the following topics:
 - **Continuations** (Lecture 14 & 15)
 - **First-Class Continuations** (Lecture 16)
 - **Compiling with continuations** (Lecture 17)
- A **continuation** represents the **rest of the computation**.
 - Continuation Passing Style (CPS)
 - Interpreter of FAE in CPS
- So far, we have defined **big-step operational (natural) semantics** for our languages.

- We will learn about **continuations** with the following topics:
 - **Continuations** (Lecture 14 & 15)
 - **First-Class Continuations** (Lecture 16)
 - **Compiling with continuations** (Lecture 17)
- A **continuation** represents the **rest of the computation**.
 - Continuation Passing Style (CPS)
 - Interpreter of FAE in CPS
- So far, we have defined **big-step operational (natural) semantics** for our languages.
- In this lecture, we define **small-step operational semantics** of FAE using **continuations** before moving on to **first-class continuations**.

1. Recall

Recall: Interpreter of FAE in CPS

Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE

Number

Addition

Multiplication

Identifier Lookup

Function Definition

Function Application

Semantic Equivalence

Example

3. First-Order Representations of Continuations

1. Recall

Recall: Interpreter of FAE in CPS

Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE

Number

Addition

Multiplication

Identifier Lookup

Function Definition

Function Application

Semantic Equivalence

Example

3. First-Order Representations of Continuations

In the previous lecture, we represented the **continuation** of each expression in the interpreter of FAE as a **function** and implemented the interpreter in **continuation passing style (CPS)**:

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

In the previous lecture, we represented the **continuation** of each expression in the interpreter of FAE as a **function** and implemented the interpreter in **continuation passing style (CPS)**:

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

Then, how can we define the **continuations** for the semantics of FAE?

Values	$\mathbb{V} \ni v ::= n$	(NumV)
	$\mid \langle \lambda x.e, \sigma \rangle$	(CloV)
Environments	$\sigma \in \mathbb{X} \xrightarrow{\text{fin}} \mathbb{V}$	(Env)
Continuations	$= ???$	

The derivation of **big-step operational (natural) semantics** has a **tree** structure, where each derivation describes the **whole evaluation** of an expression:

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2} \\
 \text{ADD} \frac{}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \text{MUL} \frac{\text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{}{\emptyset \vdash 1 + 2 \Rightarrow 3}}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

but the derivation of **small-step operational semantics** is **linear** and describes **each step** of an expression:

$$5 * (1 + 2) \rightarrow 5 * 3 \rightarrow 15$$

The derivation of **big-step operational (natural) semantics** has a **tree** structure, where each derivation describes the **whole evaluation** of an expression:

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

but the derivation of **small-step operational semantics** is **linear** and describes **each step** of an expression:

$$5 * (1 + 2) \rightarrow 5 * 3 \rightarrow 15$$

It is possible but non-trivial to represent **continuations** in the **big-step operational (natural) semantics**.

The derivation of **big-step operational (natural) semantics** has a **tree** structure, where each derivation describes the **whole evaluation** of an expression:

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

but the derivation of **small-step operational semantics** is **linear** and describes **each step** of an expression:

$$5 * (1 + 2) \rightarrow 5 * 3 \rightarrow 15$$

It is possible but non-trivial to represent **continuations** in the **big-step operational (natural) semantics**.

Let's define the **small-step operational semantics** of FAE to represent **continuations**.

1. Recall

Recall: Interpreter of FAE in CPS

Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE

Number

Addition

Multiplication

Identifier Lookup

Function Definition

Function Application

Semantic Equivalence

Example

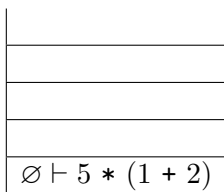
3. First-Order Representations of Continuations

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

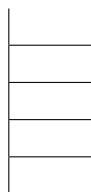
Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:



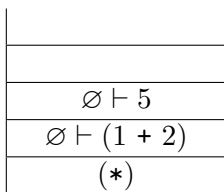
Continuation



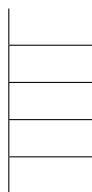
Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:



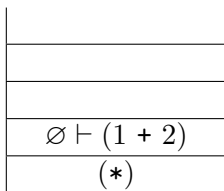
Continuation



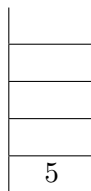
Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:



Continuation



Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:

$\emptyset \vdash 1$
$\emptyset \vdash 2$
$(+)$
$(*)$

Continuation

5

Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:

$\emptyset \vdash 2$
$(+)$
$(*)$

Continuation

1
5

Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:

(+)
(*)

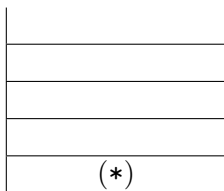
Continuation

2
1
5

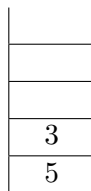
Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:



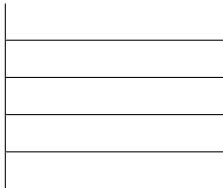
Continuation



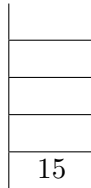
Value Stack

$$\begin{array}{c}
 \text{NUM} \frac{}{\emptyset \vdash 5 \Rightarrow 5} \quad \text{ADD} \frac{\text{NUM} \frac{}{\emptyset \vdash 1 \Rightarrow 1} \quad \text{NUM} \frac{}{\emptyset \vdash 2 \Rightarrow 2}}{\emptyset \vdash 1 + 2 \Rightarrow 3} \\
 \hline
 \text{MUL} \frac{}{\emptyset \vdash 5 * (1 + 2) \Rightarrow 15}
 \end{array}$$

Let's describe what happens **each step** of the evaluation of the expression $5 * (1 + 2)$ using **continuation** and a **value stack**:



Continuation



Value Stack

- **Big-step operational (natural) semantics:**

$$\sigma \vdash e \Rightarrow v$$

- **Small-step operational semantics:**

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle$$

where $\rightarrow \in (\mathbb{K} \times \mathbb{S}) \times (\mathbb{K} \times \mathbb{S})$ is a **reduction relation** between **states**.

$$\begin{array}{lcl} \text{Continuations} & \mathbb{K} \ni \kappa ::= & \square \\ & & | (\sigma \vdash e) :: \kappa \\ & & | (+) :: \kappa \\ & & | (*) :: \kappa \\ & & | (@) :: \kappa \end{array}$$

$$\text{Value Stacks} \quad \mathbb{S} \ni s ::= \blacksquare \mid v :: s$$

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Num(n) => k(NumV(n))
```

$$\boxed{\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle}$$

$$\text{Num} \quad \langle (\sigma \vdash n) :: \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel n :: s \rangle$$

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
...
case Add(l, r) =>
  interpCPS(l, env, {
    lv => interpCPS(r, env, {
      rv => k(numAdd(lv, rv))
    })
  })
})
```

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle$$

$$\text{Add}_1 \quad \langle (\sigma \vdash e_1 + e_2) :: \kappa \parallel s \rangle \rightarrow \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (+) :: \kappa \parallel s \rangle$$

$$\text{Add}_2 \quad \langle (+) :: \kappa \parallel n_2 :: n_1 :: s \rangle \rightarrow \langle \kappa \parallel (n_1 + n_2) :: s \rangle$$

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
...
case Mul(l, r) =>
  interpCPS(l, env, {
    lv => interpCPS(r, env, {
      rv => k(numMul(lv, rv))
    })
  })
})
```

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle$$

$$\text{Mul}_1 \quad \langle (\sigma \vdash e_1 * e_2) :: \kappa \parallel s \rangle \rightarrow \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (*) :: \kappa \parallel s \rangle$$

$$\text{Mul}_2 \quad \langle (*) :: \kappa \parallel n_2 :: n_1 :: s \rangle \rightarrow \langle \kappa \parallel (n_1 \times n_2) :: s \rangle$$

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
  ...
  case Id(x) => k(lookupId(x, env))
```

$$\boxed{\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle}$$

$$\text{Id} \quad \langle (\sigma \vdash x) :: \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel \sigma(x) :: s \rangle$$

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
...
case Fun(p, b) => k(CloV(p, b, env))
```

$$\boxed{\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle}$$

$$\text{Fun} \quad \langle (\sigma \vdash \lambda x.e) :: \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel \langle \lambda x.e, \sigma \rangle :: s \rangle$$

```
def interpCPS(expr: Expr, env: Env, k: Cont): Value = expr match
...
case App(f, e) => interpCPS(f, env, v => v match
  case CloV(p, b, fenv) =>
    interpCPS(e, env, v => {
      interpCPS(b, fenv + (p -> v), k)
    })
  case v => error(s"not a function: ${v.str}")
)
```

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle$$

$$\text{App}_1 \langle (\sigma \vdash e_1(e_2)) :: \kappa \parallel s \rangle \rightarrow \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \parallel s \rangle$$

$$\text{App}_2 \langle (@) :: \kappa \parallel v_2 :: \langle \lambda x. e, \sigma \rangle :: s \rangle \rightarrow \langle (\sigma[x \mapsto v_2] \vdash e) :: \kappa \parallel s \rangle$$

- The **reflexive transitive closure** $(\rightarrow^*)$ of $(\rightarrow)$:

$$\langle \kappa \parallel s \rangle \rightarrow^* \langle \kappa \parallel s \rangle$$

$$\frac{\langle \kappa \parallel s \rangle \rightarrow^* \langle \kappa' \parallel s' \rangle \quad \langle \kappa' \parallel s' \rangle \rightarrow \langle \kappa'' \parallel s'' \rangle}{\langle \kappa \parallel s \rangle \rightarrow^* \langle \kappa'' \parallel s'' \rangle}$$

- The **reflexive transitive closure** $(\rightarrow^*)$ of $(\rightarrow)$:

$$\langle \kappa \parallel s \rangle \rightarrow^* \langle \kappa \parallel s \rangle$$

$$\frac{\langle \kappa \parallel s \rangle \rightarrow^* \langle \kappa' \parallel s' \rangle \quad \langle \kappa' \parallel s' \rangle \rightarrow \langle \kappa'' \parallel s'' \rangle}{\langle \kappa \parallel s \rangle \rightarrow^* \langle \kappa'' \parallel s'' \rangle}$$

- The **semantic equivalence** between natural and small-step semantics:

$$\emptyset \vdash e \Rightarrow v \quad \Longleftrightarrow \quad \langle (\emptyset \vdash e) :: \square \parallel \blacksquare \rangle \rightarrow^* \langle \square \parallel v :: \blacksquare \rangle$$

More generally, the following are equivalent:

$$\sigma \vdash e \Rightarrow v \quad \Longleftrightarrow \quad \langle (\sigma \vdash e) :: \kappa \parallel s \rangle \rightarrow^* \langle \kappa \parallel v :: s \rangle$$

for all $\sigma \in \mathbb{X} \xrightarrow{\text{fin}} \mathbb{V}$, $e \in \mathbb{E}$, $v \in \mathbb{V}$, $\kappa \in \mathbb{K}$, and $s \in \mathbb{S}$.

Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

Example

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \quad \parallel \blacksquare \quad \rangle$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\begin{array}{l}
 (\text{App}_1) \quad \langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
 \xrightarrow{\quad} \langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square \quad || \quad \blacksquare \quad \rangle
 \end{array}$$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\begin{array}{lcl}
 (\text{App}_1) & \langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square & || \blacksquare \rangle \\
 \xrightarrow{\quad} & \langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square & || \blacksquare \rangle \\
 (\text{Fun}) & & \\
 \xrightarrow{\quad} & \langle (\emptyset \vdash 2) :: (@) :: \square & || \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \rangle
 \end{array}$$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\begin{array}{ll}
 (\text{App}_1) & \langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 (\text{Fun}) & \langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square \quad || \quad \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 (\text{Num}) & \langle (\emptyset \vdash 2) :: (@) :: \square \quad || \quad \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 & \langle (@) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \quad \rangle
 \end{array}$$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\begin{array}{ll}
 (\text{App}_1) & \langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \quad || \quad \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 (\text{Fun}) & \langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square \quad || \quad \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 (\text{Num}) & \langle (\emptyset \vdash 2) :: (@) :: \square \quad || \quad \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 (\text{App}_2) & \langle (@) :: \square \quad || \quad 2 :: \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \quad \rangle \\
 \xrightarrow{\quad} & \\
 & \langle ([x \mapsto 2] \vdash (1 + x)) :: \square \quad || \quad \blacksquare \quad \rangle
 \end{array}$$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

$$\begin{array}{llll}
 (\text{App}_1) & \langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square & || & \blacksquare \rangle \\
 \rightarrow & \langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square & || & \blacksquare \rangle \\
 (\text{Fun}) & \langle (\emptyset \vdash 2) :: (@) :: \square & || & \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \rangle \\
 \rightarrow & \langle (@) :: \square & || & 2 :: \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \rangle \\
 (\text{App}_2) & \langle ([x \mapsto 2] \vdash (1 + x)) :: \square & || & \blacksquare \rangle \\
 \rightarrow & \langle ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square & || & \blacksquare \rangle \\
 (\text{Add}_1) & & &
 \end{array}$$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

	$\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square$	$\parallel$	■	$\rangle$
(App ₁) →	$\langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square$	$\parallel$	■	$\rangle$
(Fun) →	$\langle (\emptyset \vdash 2) :: (@) :: \square$	$\parallel$	$\langle \lambda x.(1 + x), \emptyset \rangle ::$	■ $\rangle$
(Num) →	$\langle (@) :: \square$	$\parallel$	$2 :: \langle \lambda x.(1 + x), \emptyset \rangle ::$	■ $\rangle$
(App ₂) →	$\langle ([x \mapsto 2] \vdash (1 + x)) :: \square$	$\parallel$	■	$\rangle$
(Add ₁) →	$\langle ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square$	$\parallel$	■	$\rangle$
(Num) →	$\langle ([x \mapsto 2] \vdash x) :: (+) :: \square$	$\parallel$	$1 ::$	■ $\rangle$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

	$\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square$	$\parallel$	■	$\rangle$
(App ₁) →	$\langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square$	$\parallel$	■	$\rangle$
(Fun) →	$\langle (\emptyset \vdash 2) :: (@) :: \square$	$\parallel$	$\langle \lambda x.(1 + x), \emptyset \rangle ::$	■ $\rangle$
(Num) →	$\langle (@) :: \square$	$\parallel$	2 :: $\langle \lambda x.(1 + x), \emptyset \rangle ::$	■ $\rangle$
(App ₂) →	$\langle ([x \mapsto 2] \vdash (1 + x)) :: \square$	$\parallel$	■	$\rangle$
(Add ₁) →	$\langle ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square$	$\parallel$	■	$\rangle$
(Num) →	$\langle ([x \mapsto 2] \vdash x) :: (+) :: \square$	$\parallel$	1 ::	■ $\rangle$
(Id) →	$\langle (+) :: \square$	$\parallel$	2 :: 1 ::	■ $\rangle$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

	$\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square$	$\parallel$	■	$\rangle$
(App ₁) →	$\langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square$	$\parallel$	■	$\rangle$
(Fun) →	$\langle (\emptyset \vdash 2) :: (@) :: \square$	$\parallel$	$\langle \lambda x.(1 + x), \emptyset \rangle ::$	■ $\rangle$
(Num) →	$\langle (@) :: \square$	$\parallel$	2 :: $\langle \lambda x.(1 + x), \emptyset \rangle ::$	■ $\rangle$
(App ₂) →	$\langle ([x \mapsto 2] \vdash (1 + x)) :: \square$	$\parallel$	■	$\rangle$
(Add ₁) →	$\langle ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square$	$\parallel$	■	$\rangle$
(Num) →	$\langle ([x \mapsto 2] \vdash x) :: (+) :: \square$	$\parallel$	1 ::	■ $\rangle$
(Id) →	$\langle (+) :: \square$	$\parallel$	2 :: 1 ::	■ $\rangle$
(Add ₂) →	$\langle \square$	$\parallel$	3 ::	■ $\rangle$

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

	$\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \rangle$	$\parallel$	$\blacksquare$	$\rangle$
(App ₁)	$\xrightarrow{\quad} \langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square \rangle$	$\parallel$	$\blacksquare$	$\rangle$
(Fun)	$\xrightarrow{\quad} \langle (\emptyset \vdash 2) :: (@) :: \square \rangle$	$\parallel$	$\langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare$	$\rangle$
(Num)	$\xrightarrow{\quad} \langle (@) :: \square \rangle$	$\parallel$	$2 :: \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare$	$\rangle$
(App ₂)	$\xrightarrow{\quad} \langle ([x \mapsto 2] \vdash (1 + x)) :: \square \rangle$	$\parallel$	$\blacksquare$	$\rangle$
(Add ₁)	$\xrightarrow{\quad} \langle ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square \rangle$	$\parallel$	$\blacksquare$	$\rangle$
(Num)	$\xrightarrow{\quad} \langle ([x \mapsto 2] \vdash x) :: (+) :: \square \rangle$	$\parallel$	$1 :: \blacksquare$	$\rangle$
(Id)	$\xrightarrow{\quad} \langle (+) :: \square \rangle$	$\parallel$	$2 :: 1 :: \blacksquare$	$\rangle$
(Add ₂)	$\xrightarrow{\quad} \langle \square \rangle$	$\parallel$	$3 :: \blacksquare$	$\rangle$

Thus, $\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \parallel \blacksquare \rangle \rightarrow^* \langle \square \parallel 3 :: \blacksquare \rangle$.

Let's interpret the expression $(\lambda x.(1 + x))(2)$:

	$\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \rangle$	$\parallel \blacksquare \rangle$
(App ₁) →	$\langle (\emptyset \vdash \lambda x.(1 + x)) :: (\emptyset \vdash 2) :: (@) :: \square \rangle$	$\parallel \blacksquare \rangle$
(Fun) →	$\langle (\emptyset \vdash 2) :: (@) :: \square \rangle$	$\parallel \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \rangle$
(Num) →	$\langle (@) :: \square \rangle$	$\parallel 2 :: \langle \lambda x.(1 + x), \emptyset \rangle :: \blacksquare \rangle$
(App ₂) →	$\langle ([x \mapsto 2] \vdash (1 + x)) :: \square \rangle$	$\parallel \blacksquare \rangle$
(Add ₁) →	$\langle ([x \mapsto 2] \vdash 1) :: ([x \mapsto 2] \vdash x) :: (+) :: \square \rangle$	$\parallel \blacksquare \rangle$
(Num) →	$\langle ([x \mapsto 2] \vdash x) :: (+) :: \square \rangle$	$\parallel 1 :: \blacksquare \rangle$
(Id) →	$\langle (+) :: \square \rangle$	$\parallel 2 :: 1 :: \blacksquare \rangle$
(Add ₂) →	$\langle \square \rangle$	$\parallel 3 :: \blacksquare \rangle$

Thus, $\langle (\emptyset \vdash (\lambda x.(1 + x))(2)) :: \square \parallel \blacksquare \rangle \rightarrow^* \langle \square \parallel 3 :: \blacksquare \rangle$.

It means that the evaluation result of $(\lambda x.(1 + x))(2)$ is 3.

1. Recall

Recall: Interpreter of FAE in CPS

Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE

Number

Addition

Multiplication

Identifier Lookup

Function Definition

Function Application

Semantic Equivalence

Example

3. First-Order Representations of Continuations

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

How can we define continuations if we want to implement the interpreter for FAE in CPS using a **non-functional** language (e.g., C)?

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

How can we define continuations if we want to implement the interpreter for FAE in CPS using a **non-functional** language (e.g., C)?

Let's define the continuations as **data structures** (e.g., algebraic data types) in such languages.

In our new implementation for FAE using CPS, we define **continuations** as **first-class functions** in Scala.

```
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
type Env = Map[String, Value]
type Cont = Value => Value

def interpCPS(expr: Expr, env: Env, k: Cont): Value = ...
```

How can we define continuations if we want to implement the interpreter for FAE in CPS using a **non-functional** language (e.g., C)?

Let's define the continuations as **data structures** (e.g., algebraic data types) in such languages.

We call them the **first-order representations of continuations**.

```
enum Cont:
  case EmptyK
  case EvalK(env: Env, expr: Expr, k: Cont)
  case AddK(k: Cont)
  case MulK(k: Cont)
  case AppK(k: Cont)

type Stack = List[Value]
```

Continuations $\mathbb{K} \ni \kappa ::= \square \quad (\text{EmptyK})$
 $\quad \quad \quad | (\sigma \vdash e) :: \kappa \quad (\text{EvalK})$
 $\quad \quad \quad | (+) :: \kappa \quad (\text{AddK})$
 $\quad \quad \quad | (*) :: \kappa \quad (\text{MulK})$
 $\quad \quad \quad | (@) :: \kappa \quad (\text{AppK})$

Value Stacks $\mathbb{S} \ni s ::= \blacksquare \mid v :: s \quad (\text{List[Value]})$

We define a `step` function that takes a state $\langle \kappa \parallel s \rangle$ and **steps** it to another state $\langle \kappa' \parallel s' \rangle$ using the reduction relation $\rightarrow$ we defined before:

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa' \parallel s' \rangle$$

```
def step(k: Cont, s: Stack): (Cont, Stack) = ???
```

We define a **step** function that takes a state $\langle \kappa \parallel s \rangle$ and **steps** it to another state $\langle \kappa' \parallel s' \rangle$ using the reduction relation $\rightarrow$ we defined before:

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa' \parallel s' \rangle$$

```
def step(k: Cont, s: Stack): (Cont, Stack) = ???
```

And the **evalK** function **iteratively steps** the state until it reaches the empty continuation $\square$ and returns the single value in the value stack:

```
def evalK(str: String): String =
  import Cont.*
  def aux(k: Cont, s: Stack): Value = step(k, s) match
    case (EmptyK, List(v)) => v
    case (k, s)             => aux(k, s)
  aux(EvalK(Map.empty, Expr(str), EmptyK), List.empty).str
```

$$\langle (\emptyset \vdash e) :: \square \parallel \blacksquare \rangle \rightarrow^* \langle \square \parallel v :: \blacksquare \rangle$$

```
def step(k: Cont, s: Stack): (Cont, Stack) = (k, s) match
  case (EvalK(env, expr, k), s) => expr match
    ...
    case Add(l, r) => (EvalK(env, l, EvalK(env, r, AddK(k))), s)
  ...
  case (AddK(k), r :: l :: s) => (k, numAdd(l, r) :: s)
  ...
```

$$\langle \kappa \parallel s \rangle \rightarrow \langle \kappa \parallel s \rangle$$

$$\text{Add}_1 \quad \langle (\sigma \vdash e_1 + e_2) :: \kappa \parallel s \rangle \rightarrow \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (+) :: \kappa \parallel s \rangle$$

$$\text{Add}_2 \quad \langle (+) :: \kappa \parallel n_2 :: n_1 :: s \rangle \rightarrow \langle \kappa \parallel (n_1 + n_2) :: s \rangle$$

Similarly, we can define the step function for the other cases.

1. Recall

Recall: Interpreter of FAE in CPS

Recall: Big-Step vs. Small-Step Semantics

2. Small-Step Semantics of FAE

Number

Addition

Multiplication

Identifier Lookup

Function Definition

Function Application

Semantic Equivalence

Example

3. First-Order Representations of Continuations

<https://github.com/ku-plrg-classroom/docs/tree/main/cose212/fae-cps>

- Please see above document on GitHub:
 - Implement `interpCPS` function.
 - Implement `step` function.
- It is just an exercise, and you **don't need to submit** anything.
- However, some exam questions might be related to this exercise, so we **strongly recommend** solving it by yourself.

- First-Class Continuations

Jihyeok Park

`jihyeok_park@korea.ac.kr`

`https://plrg.korea.ac.kr`